Not Approved. This proposal was explored using Claud. There are many wrong info and explanation in regards to real-life scenario. This is an idea only and it is better to just take a bit of inspiration from here and not follow it.

Project Proposal: On-Chip KV Reuse and Shared-Prefix Attention for JEPA Architectures

Prepared by Sumaiya Tarannum Noor; ID-2425410650 · October 2026 · Draft 2 (JEPA version)

1. Summary

Our first proposal targeted autoregressive decoding, where one query row reads a growing KV cache from HBM. Standard JEPA models do not decode that way, so we are reframing the project. We can study where FlashAttention-style tiling, KV reuse and a hot on-chip window can remove real cost in JEPA predictors, starting with the one JEPA component that genuinely has a KV cache: the block-causal action-conditioned predictor in V-JEPA 2-AC. We can build an IO and compute model, implement a shared-prefix attention kernel, and measure latency, memory and accuracy against baselines.

2. Why the Decode Proposal Does Not Transfer Directly

- **Bidirectional attention.** The predictors in I-JEPA, V-JEPA and VL-JEPA are non-autoregressive and use bidirectional attention. Adding or changing a token changes every token's state after the first layer, so a cache of past K/V goes stale.
- **Training changes the weights.** During pre-training the projection weights change every step, so cached K/V or cached attention statistics cannot be reused across steps.
- **Prefill-like shapes.** A JEPA predictor predicts many masked or next-frame tokens in one forward pass. That means many query rows per call (large B_r), which is compute-bound rather than the memory-bound, one-row decode regime of the first proposal.

So the honest starting point is that the memory-bound decode argument is weak for most of JEPA. The project has to find the places where reuse is both valid and valuable.

3. Where Reuse Is Valid in JEPA

3.1 Primary target: V-JEPA 2-AC planning (block-causal)

V-JEPA 2-AC is a 24-layer, width-1024 predictor with block-causal attention, where each token attends to tokens, actions and states from its own frame and earlier ones. Because the attention is

causal across frames, K/V of past frames are valid to cache. Planning with a model-predictive loop (reported as CEM with hundreds of candidate action sequences over several refinement iterations) evaluates many candidates that share the same observed history. Their history K/V are identical; only the candidate action tokens and predicted-frame tokens differ.

The expected gain is shared-prefix reuse: compute the history K/V once, store it once, and let all candidates read it. This removes redundant projection, MLP and attention work on the history and removes N duplicate copies of it in memory. It is a compute and memory-capacity gain, not a bandwidth gain, because each candidate already contributes many query rows.

3.2 Secondary target: short-query, long-KV attention

Some JEPA-related attention calls have few query rows against many visual tokens: the text-query tokens attending to visual tokens in the VL-JEPA predictor, and the learned query tokens of an attentive probe over frozen V-JEPA 2 features. These are closer to the memory-bound regime of the first proposal. We can use the IO model and split-KV merging there, at inference time only, and only if profiling shows they matter.

3.3 Exploratory target: prefix-style masking in I-JEPA and V-JEPA

In I-JEPA-style training, several target blocks are predicted from the same context. If context tokens did not attend to the mask tokens, the context K/V would be shareable across all target blocks. This changes the model, so we can treat it as an experiment that needs fine-tuning and an accuracy comparison against the standard predictor.

4. Research Questions

- **RQ1:** How much compute, memory and latency does shared-prefix K/V reuse save in V-JEPA 2-AC planning as the number of candidates N, history length and horizon vary?
- **RQ2:** Does the compute and IO model predict the measured latency, and where does it fail?
- **RQ3:** In the short-query, long-KV calls (VL-JEPA predictor, attentive probe), how memory-bound are they in practice, and does split-KV with the online-softmax merge help?
- **RQ4:** Does prefix-style masking in a JEPA predictor keep representation quality while enabling context reuse across target blocks?

5. Proposed Approach

5.1 Analytical model

We can model per-layer cost for P shared prefix tokens, N candidates and Q new query tokens per candidate. Without sharing, projection and MLP work scales as N * (P + Q); with sharing it scales as P + N * Q. Memory for the prefix K/V drops from N copies to one. For attention we can reuse the tile model: HBM traffic for the prefix is about ceil(N * Q / B_r) * 2 * P * d * b

bytes, with B_r the Q-tile height and b bytes per element. We can estimate cache size per frame as 2 * d_model * b * layers * tokens per frame, and fill in the exact token counts from the released configuration.

5.2 Shared-prefix attention kernel

We can write a FlashAttention-style kernel (Triton first, CUDA if profiling demands it) in which all candidate queries attend to one shared prefix K/V in HBM and to their own private tokens. It merges the two parts with the online-softmax combine on (m, l, O), which is exact because the combine operator is associative.

5.3 Planner integration

We can integrate the kernel into the open V-JEPA 2-AC predictor and keep a prefix cache that extends by one frame per control step in the receding-horizon loop, so the history is never recomputed.

5.4 Optional compressed hot window

If the prefix cache becomes large, we can test int8 or int4 quantization of old-frame K/V and a window of the most recent frames, and report the planning-success impact separately from the exact path.

6. Evaluation Plan

- **Models:** the released V-JEPA 2-AC predictor with its frozen encoder; an open VL-JEPA predictor for the secondary target.
- **Baselines:** the original predictor with standard FlashAttention and no prefix sharing; a naive batched-candidate baseline.
- **Metrics:** planning latency per action, GPU memory for the candidate batch, FLOPs, HBM bytes moved (Nsight Compute), output equality against the baseline (max absolute error), and planning success where a simulator or the released benchmark is available.
- **Sweeps:** N, history frames, horizon, precision, window size.
- **Correctness:** the exact path must match the baseline predictor outputs within floating-point tolerance before we report any timing.

7. Milestones

1. **Phase 1 (weeks 1-2):** profile the released predictors and confirm token counts, horizon, candidate counts and which attention calls dominate time.
2. **Phase 2 (weeks 3-4):** build the analytical model and the shared-prefix kernel, and validate exactness.
3. **Phase 3 (weeks 5-6):** integrate with the planner loop, measure RQ1 and RQ2.
4. **Phase 4 (weeks 7-8):** study the short-query cases (RQ3) and, if time allows, the prefix-masking experiment (RQ4).

5. **Phase 5 (weeks 9-10):** write up results, including negative findings.

8. Risks and Mitigations

- **The history is a small part of the planner's cost.** Phase 1 profiling comes first, and if the saving is small we can report that result honestly.
- **Few candidates or a short horizon limit the gain.** We can map the gain as a function of N and history length instead of claiming a single number.
- **Prefix-masking hurts representation quality.** It stays an exploratory branch and does not affect the exact planning path.
- **Hardware dependence.** We can report numbers per GPU model.
- **Details we have not verified.** Token counts per frame, horizon and candidate counts come from secondary sources and must be checked against the paper and released code before they enter any claim.

9. Expected Contributions

1. A cost model for attention in JEPA predictors that separates the memory-bound short-query case from the compute-bound prefill-like case.
2. A shared-prefix attention kernel and prefix cache for block-causal JEPA world-model planning.
3. A measured account of where KV reuse helps JEPA architectures and where it does not.

10. Claims Policy

We can describe only valid cache reuse: causal prefixes and shared inputs. We can leave out any claim of KV caching for bidirectional predictors or for training. We can state speedups only after measurement and report exactness error with every timing.